



Instituto Tecnológico de Mazatlán

Interrupciones externas y buffer circular

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Hernández Barrón

Alumnos:

Rosa Guadalupe Romero Navarro

Javier Ramón Quintero Valdez

Gabriel Alejandro Barrón Sarabia

Reporte de práctica: Interrupciones externas y buffer circular

1. Descripción del proyecto

Esta práctica consiste en desarrollar un sistema capaz de detectar y registrar eventos generados de manera rápida e impredecible mediante un pulsador, utilizando una interrupción externa en Arduino UNO R4 WiFi.

El pulsador representa el sensor de una banda transportadora que detecta el paso de piezas. Cada vez que se presiona el botón, el Arduino registra el instante en que ocurrió el evento y lo almacena temporalmente en un buffer circular.

Al mismo tiempo, el Arduino ejecuta una animación en la matriz de LEDs integrada en la placa. El objetivo es demostrar que los eventos generados por el pulsador pueden registrarse aunque el programa se encuentre realizando otra tarea.

2. Objetivos

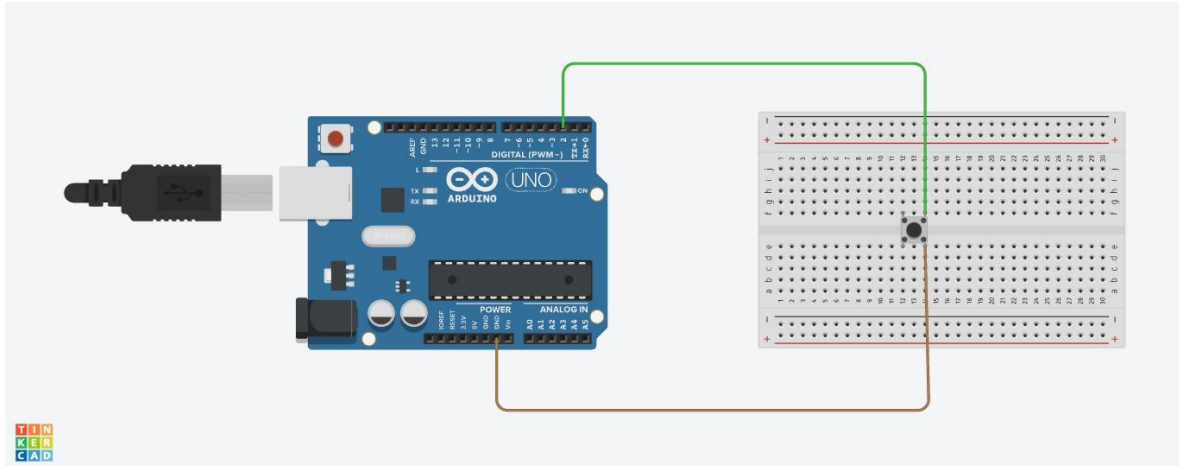
- Comprender el funcionamiento de las interrupciones externas en Arduino.
- Utilizar un pulsador como simulación de un sensor de piezas.
- Registrar eventos mediante una rutina de interrupción.
- Implementar un buffer circular de tamaño fijo para almacenar eventos.
- Aplicar un filtro de rebote por software dentro de la interrupción.
- Utilizar `millis()` para realizar una animación sin bloquear la ejecución.
- Procesar posteriormente los eventos registrados desde el `loop()`.
- Comprender el funcionamiento de los contadores de escritura y lectura para diferenciar entre un buffer vacío y uno lleno.

3. Herramientas y material utilizado

- Arduino UNO R4 WiFi.
- Arduino IDE.
- Matriz de LEDs integrada en el Arduino UNO R4 WiFi.
- Pulsador.
- Cables de conexión.
- Protoboard.
- Computadora.

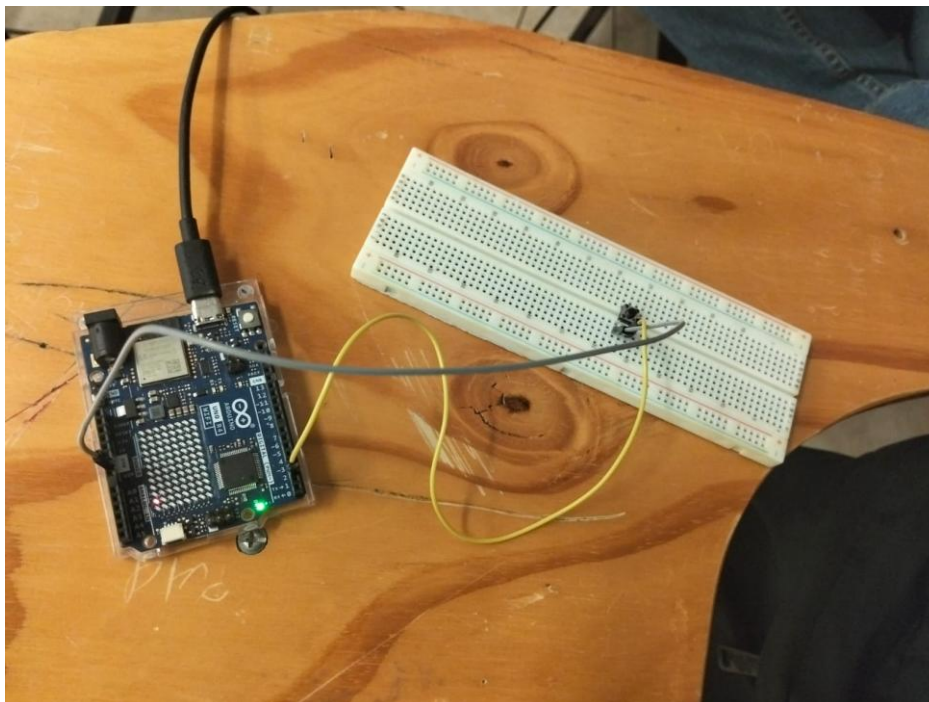
4. Diagrama

El diagrama representa la conexión utilizada para el pulsador y el funcionamiento general del sistema.



El pulsador se conecta al pin 2 del Arduino y al GND. Se utiliza la configuración `INPUT_PULLUP`, por lo que el pin permanece en estado HIGH mientras el botón no está presionado y cambia a LOW al realizarse una pulsación.

El pin 2 se utiliza como entrada de interrupción externa para detectar los cambios de estado generados por el pulsador.



5. Metodología

Primero se configuró el Arduino UNO R4 WiFi para utilizar su matriz de LEDs integrada y el pulsador como entrada para detectar eventos.

Posteriormente se configuró el pin 2 como entrada con resistencia pull-up interna y se estableció una interrupción externa para detectar el flanco de bajada generado al presionar el pulsador.

Cada vez que se genera una interrupción, la función encargada de atender el evento obtiene el instante en que ocurrió y lo almacena en un buffer circular. Debido a que esta función debe ejecutarse rápidamente, solamente realiza las operaciones necesarias para registrar el evento y posteriormente devuelve el control al programa principal.

Para evitar que una sola pulsación genere varios registros debido al rebote mecánico del botón, se implementó un filtro de rebote basado en microsegundos.

Mientras se registran los eventos del pulsador, el programa principal mantiene una animación en la matriz de LEDs. Esta animación utiliza `millis()` para determinar cuándo debe avanzar al siguiente cuadro, evitando detener la ejecución mediante pausas prolongadas.

Finalmente, el `loop()` revisa los eventos que fueron registrados por la interrupción y los procesa uno por uno. Cada evento procesado incrementa el contador de piezas y muestra en el monitor serie el número de pieza y el instante en que fue detectada.

6. Explicación del funcionamiento

El sistema utiliza una interrupción externa para responder inmediatamente a la pulsación del botón. Cuando el Arduino detecta un cambio de HIGH a LOW en el pin 2, interrumpe momentáneamente la ejecución normal y llama a la función encargada de registrar el evento.

La función de interrupción obtiene el tiempo actual mediante `micros()` y comprueba si ha transcurrido el tiempo mínimo establecido para evitar el rebote. Si el evento es válido, guarda el instante de detección en el buffer circular y aumenta el contador de eventos escritos.

El buffer utilizado tiene un tamaño de 16 posiciones. Para recorrerlo de manera circular se utiliza el operador `%`, permitiendo regresar al principio del arreglo cuando se llega a su última posición.

Para distinguir entre eventos escritos y eventos ya procesados se utilizan dos contadores. `escritos` indica cuántos eventos han sido registrados por la interrupción, mientras que `leídos` indica cuántos eventos han sido procesados por el `loop()`.

De esta manera, la interrupción y el programa principal trabajan a diferentes velocidades. La interrupción registra rápidamente los eventos cuando ocurren, mientras que el `loop()` los procesa posteriormente sin necesidad de detener la animación.

7. Animación de la matriz de LEDs

La matriz de LEDs integrada en el Arduino UNO R4 WiFi muestra una animación de un punto que recorre sus columnas y avanza por las diferentes filas.

La animación utiliza un intervalo de 120 milisegundos entre cada movimiento. Para controlar este intervalo se utiliza `millis()`, permitiendo que el Arduino compruebe constantemente si corresponde avanzar al siguiente cuadro sin utilizar una espera que bloquee el programa.

De esta manera, la animación continúa funcionando mientras el Arduino atiende y registra las interrupciones generadas por el pulsador.

8. Análisis de resultados

Durante las pruebas se pudo observar que el pulsador genera una interrupción al producirse el cambio de estado correspondiente. El evento es registrado rápidamente y posteriormente procesado por el programa principal.

El uso del buffer circular permite separar el momento en que ocurre el evento del momento en que el programa puede procesarlo. Esto resulta importante cuando el pulsador se presiona mientras el Arduino se encuentra realizando la animación de la matriz de LEDs.

El filtro de rebote evita que una sola pulsación mecánica sea registrada varias veces debido a las pequeñas variaciones eléctricas que ocurren durante el contacto del botón.

También se implementó un mecanismo para detectar el desbordamiento del buffer. Si se generan más eventos de los que pueden almacenarse antes de que el `loop()` los procese, el programa identifica esta situación y muestra un mensaje de advertencia en el monitor serie.

Las pruebas permiten observar que la interrupción externa puede utilizarse para detectar eventos que ocurren de manera impredecible, mientras que el programa principal continúa realizando otras tareas.

9. Conclusiones

Gabriel

En esta práctica comprendí cómo las interrupciones externas permiten que el Arduino detecte eventos de manera inmediata mientras realiza otras tareas. También aprendí la

utilidad del buffer circular para almacenar temporalmente los eventos del pulsador y procesarlos posteriormente sin detener la animación de la matriz de LEDs.

Javier

Durante la práctica aprendí la importancia de mantener una rutina de interrupción rápida y sencilla. El uso del `debounce` permitió evitar que un solo pulsado del botón fuera registrado varias veces, mientras que los contadores `escritos` y `leídos` facilitaron el control de los eventos almacenados en el buffer.

Rosa

Esta práctica me permitió comprender cómo realizar varias tareas al mismo tiempo utilizando una programación no bloqueante. La animación de la matriz continúa funcionando mientras el Arduino detecta y procesa las pulsaciones del botón, mostrando cómo las interrupciones, `millis()` y el buffer circular pueden trabajar juntos para responder a eventos sin detener el programa.